ChatGPT

# Analysis of Issue 003 – `get_staging_info` stage grouping and delta-v calculation

## Does the bug report make sense?

Yes. The behaviour described in the report is not expected and reproduces consistently in the logs. The MCP tool `get_staging_info` split each logical stage into two entries – one with engines but **no propellant** and another with propellant but **no engines**. The report also shows an implausibly high Δv (≈9.4 km/s) because the mass calculation reduced a stage's final mass to **0.1 kg**, which is physically impossible. Examination of the code and kRPC documentation shows that these anomalies are not due to user error but stem from how staging information is being calculated.

### Evidence from the logs

The attached `.jsonl` log shows that `get_staging_info({})` produced output like:

- Stage 6: `engines=3` but `prop_mass_kg=0.0`.
- Stage 5: `engines=0` but `prop_mass_kg=16 000.0`.
- Stage 4: `prop_mass_kg=8 000.0` and `m1_kg` is **0.1 kg**, which results in `delta_v_m_s=9366.03…`.

The separation of engines from their propellant clearly demonstrates the bug. The unrealistic Δv arises because `m1_kg` is clipped to `0.1` kg when the code subtracts propellant mass from the current mass and then forces a minimum mass; this inadvertently treats an entire stage as almost weightless when the propellant mass estimate exceeds the vessel mass.

### Root cause in the code

Inspection of the **krpc_utils** module in the MCP repository shows that `get_staging_info` computes masses and Δv by looping through the vessel's stages and calling helper functions:

1. `_combined_isp_and_thrust_for_stage` groups **engines by activation stage** (`part.stage`) and sums their thrust and specific impulse.
2. `_stage_prop_mass_kg` calls `vessel.resources_in_decouple_stage(stage, cumulative=False)`, which returns resources **decoupled** at a particular stage rather than the resources accessible to engines in that stage. Because each part has two staging numbers – the stage it is activated in and the stage it is decoupled in – resources attached via decouplers often have `decouple_stage = stage – 1` [1]. This mismatch causes propellant from a stage to be reported in the previous stage.
3. The loop updates the dry mass using:

```
m1 = max(0.1, m0 - prop_mass)  # ensures non-zero mass
mass_current = max(0.1, m1 - drop_mass)
```

When `prop_mass` is larger than `m0`, the result is artificially clipped to 0.1 kg. The subsequent Δv calculation, which uses the Tsiolkovsky rocket equation, therefore produces an enormous Δv because the ratio `m0 / m1` becomes extremely large.

These design decisions – using decouple stages for propellant and enforcing an arbitrary minimum mass – explain the anomalies seen in the bug report.

### Existing kRPC limitations and community discussions

kRPC itself does **not** provide a built-in function to compute per-stage Δv or a complete staging tree. Each part tracks two staging indices (activation stage and decouple stage), and kRPC exposes them separately. This leads to confusion when matching engines with the propellant available to them. A GitHub issue from 2016 (`krpc/krpc#274`) describes the same problem: engines appear in one stage while their fuel appears in the next because `vessel.resources_in_decouple_stage()` uses the decouple stage [1]. Contributors recommended avoiding `resources_in_decouple_stage` and instead using the **engine's** `propellants` property to determine what resources are reachable by that engine [2]. The `Propellant` class in kRPC provides attributes like `total_resource_available` (amount of a resource reachable by the engine, respecting fuel flow rules) and `total_resource_capacity` [3]. This approach directly answers "how much propellant can this engine burn?" and does not require guessing from decouple stages.

## Plan to address the issue

1. **Acknowledge the limitation and document it.**
2. Update the MCP documentation to explain that `get_staging_info` is an approximation because KSP and kRPC do not expose a simple per-stage Δv API. Highlight that `vessel.resources_in_decouple_stage()` returns resources decoupled at a given stage and does **not** necessarily correspond to the resources accessible to engines activated in that stage [1].
3. **Revise the propellant calculation.** Instead of using `vessel.resources_in_decouple_stage`:
4. For each engine in a stage, iterate through `engine.propellants` and sum the mass of each propellant's `total_resource_available` (converted to kg using resource densities). This ensures that only the propellant reachable by those engines is counted. It also naturally handles cross-feed rules and radial boosters, which `resources_in_decouple_stage` cannot.
5. To compute dry mass after burning fuel, subtract the **sum of all propellant masses actually consumed**. Avoid forcing `m1` or `mass_current` to 0.1 kg when the propellant mass estimate exceeds the current mass. Instead, if the remaining mass drops below a small threshold, treat the stage as empty and set Δv to zero for that stage.
6. **Group engines and propellant consistently.**

7. Build the stage list based on **activation stages** (`part.stage`) rather than decouple stages. For each activation stage (starting from the highest), gather all engines and compute their combined thrust and Isp. Then determine the total propellant accessible to those engines using their `propellants`. This will keep engines and fuel in the same stage entry and prevent the split seen in the logs.

8. **Handle dropped mass and decouplers.** *Dry mass* includes structural parts and empty fuel tanks that will be discarded when the stage is decoupled. Use `vessel.parts.in_decouple_stage(stage)` to find all parts jettisoned at this stage and subtract their masses from `mass_current` for the next stage. Do not subtract dry mass from the current stage until after the engine burn is finished.

9. **Test with multiple crafts and edge cases.**

10. Use the uploaded log and additional KSP vessels to verify that the revised algorithm correctly aligns engines and propellant. Check cases with radial boosters, asparagus staging, solid rocket boosters, and fuel cross-feed. Compare the computed Δv with KSP's in-game Δv indicator or tools like MechJeb to ensure reasonable agreement.

11. **Consider using approximate stage plan from kRPC.**

12. kRPC provides `vessel.stage_plan` and `vessel.approximate_stage_plan` in newer versions. These return a list of stages with total mass, dry mass and available resources, albeit with some approximations. Evaluate whether these methods produce better staging and Δv information than the current custom implementation. If they do, `get_staging_info` could wrap these functions instead of re-implementing the entire logic.

13. **Look to the community for guidance.**

14. Many kRPC users have implemented their own delta-v calculators. The recommendation from issue #274 is to use `Engine.propellants.total_resource_available` [2]. Searching forums may reveal scripts or libraries that already compute staging and delta-v. Re-using or adapting such code could save development time and ensure correctness.

## Community delta-v calculators and what we can learn from them

Several community projects for KSP and its scripting tools illustrate how to compute stage-by-stage delta-v more robustly than the current MCP approach. One of the most comprehensive examples is the **DeltaVLib.ks** library for **kOS**, which parses a vessel's parts to build per-stage dictionaries of engines, fuel and mass. Key aspects of its approach include:

1. **Build dictionaries of parts and engines by stage.** The script iterates over every part on the vessel, assigning each engine to its activation stage and assigning non-engine parts (fuel tanks, structural components, etc.) to the stage above their activation stage. It accumulates the total mass and dry mass per stage and collects a list of engines in each stage [4].
2. **Determine the range of stages an engine contributes to.** For each engine, the library calculates a **minimum stage** (the first stage the engine is active in) and a **maximum stage** (the last stage it is

active in). It then loops through all stages in that range and adds the engine's thrust and mass-flow contributions to each stage. This design automatically handles **asparagus staging** and engines that span multiple stages [5].

3. **Compute effective specific impulse and thrust per stage.** For each stage, the script sums the thrust of all engines and divides by the sum of `thrust/isp` to get the combined effective specific impulse. It also accumulates the mass-flow rate to calculate thrust-to-weight ratio (TWR) and burn time [6].
4. **Apply the rocket equation per stage.** After building the dictionaries, the script iterates through stages from lowest to highest. It maintains an accumulator of the total mass above the current stage and calculates the delta-v contribution of each stage using the Tsiolkovsky equation: `Δv = 9.81 * Isp * ln((m_acc + m_stage)/(m_acc + m_stage_dry))`. It also computes the starting and ending TWR for each stage and updates the accumulator for subsequent stages [7].

By avoiding `resources_in_decouple_stage` and instead deriving mass and engine data directly from parts, DeltaVLib avoids the split between engines and propellant. It also uses each engine's actual thrust and Isp to compute delta-v, handling complex staging (e.g., asparagus) correctly. Adapting these principles to MCP would involve building per-stage dictionaries from `vessel.parts` (as kOS does) or at least using each `Engine`'s `propellants` list to determine accessible fuel. The key lesson is to **calculate mass and propellant availability from the engines' perspective**, rather than from decouplers, and to process stages in sequence with a running total of mass.

In addition to kOS scripts, players often rely on mods like **Kerbal Engineer Redux** and **MechJeb** for accurate delta-v readouts. These tools implement similar logic: they determine which engines are active in each stage, sum the propellant those engines can reach, and apply the rocket equation to compute delta-v and burn times. While their source code is not kRPC-compatible, the algorithms mirror the principles above. Documenting these community approaches and referencing libraries such as DeltaVLib can guide users and developers seeking more precise staging calculations.

## Summary

The observed behaviour in `get_staging_info` is not intended and originates from mixing **activation stages** and **decouple stages** when calculating propellant and from forcing masses to a minimum of 0.1 kg. This results in engines and their fuel being split across stages and unrealistic Δv values. To address the issue, the tool should be rewritten to group engines and propellant based solely on activation stages and use each engine's `propellants.total_resource_available` to compute the mass of fuel actually available. Avoid artificially clipping masses to 0.1 kg, handle dry-mass drop after fuel depletion, and test the revised algorithm against multiple vessels. Clearly document the limitations and consider leveraging kRPC's built-in stage plan functions or community-provided delta-v scripts.

---

[1] [2] Confusing issue with Engine & Resource stages · Issue #274 · krpc/krpc
https://github.com/krpc/krpc/issues/274

[3] Parts — kRPC 0.5.4 documentation
https://krpc.github.io/krpc/python/api/space-center/parts.html

4 5 6 7 kOS-Launch-Scripts/libraries/DeltaVLib.ks at master · ScranchNew/kOS-Launch-Scripts · GitHub
https://github.com/ScranchNew/kOS-Launch-Scripts/blob/master/libraries/DeltaVLib.ks